|
|---|---

---|
Bugs per milestone (6mo avg)	The percentage of issues worked on per milestone that were bugs, on average, over the last 6 months.
Bug Growth Rate (6mo total)	The total percentage of bug growth over the last 6 months.
Bugs opened and closed by severity (6mo totals)	The total number of bugs opened and closed, by severity, over the last 6 months.
Open bugs by severity	The total number of open bugs, by severity, including the number and percentage past SLO.
Selected burndown plan	Summary of the burndown plan selected in the filters.
Milestone planning (bug counts to schedule)	This chart helps determine the number of bugs needed to be pulled into the milestone in order to meet the selected burndown plan. The "Open Bugs (Total)" column uses the same data from the "Open bugs by severity" chart. The "Burndown Bugs" column shows how many bugs must be pulled in per milestone, by severity, in order to start reducing the backlog. The "New Bugs" column approximates how many new bugs of each severity we can expect to have open in a given milestone. The total number of new bugs here is the minimum needed to be scheduled to not grow the backlog.
Milestone planning (bug % of scheduled issues)	This chart represents the same concept as the Milestone planning (bug counts) chart, but in percentages of the total number of MRs merged over time. The "New Bug %" column is the percentage of bugs needed to be taken up in the upcoming milestone in order to not grow the backlog, without burndown. The "Total Bug %" column is the percentage of bugs needed to be taken up in the upcoming milestone in order to burn down the backlog.
Open bugs over time	This graph visualizes the number of bugs opened over time by severity.
S1 Bug Backlog Over Time	This graph visualizes the number of open S1 bugs in the backlog over time.
S2 Bug Backlog Over Time	This graph visualizes the number of open S2 bugs in the backlog over time.
Open bugs (with severity label)	Lists open bugs with links to their respective issues, sorted by severity in descending order (severity::1 - severity::4). Each severity group is then sorted by bug age, from oldest to newest. SETs / QEMs can use

this list to put together a suggested list of bugs for their counterpart groups.
|
| Open bugs (missing severity label) | Lists open bugs that are missing a severity label.
|
| Avg MRs per milestone (6mo) | The average number of MRs opened per milestone over the last 6 months.
|
| Bugs MRs merged per bug issue closed (6mo) | The number of bug MRs merged per bug issue closed over the last 6 months.
|

Prioritization Guidelines

The following considerations can be helpful to keep in mind when determining which bugs to prioritize.

- Age (older vs. newer)
- Severity (higher vs. lower)
- Past due bugs (bugs that slipped from previous milestones, missed SLOs, etc.)
- Customer reported vs. internal (issues tagged with customer)
- "Support Priority" or "Support Efficiency" which indicate a significant number of customer tickets
- Popularity/number of users impacted
  - This will require some investigation and discussion among the quad, but there a few indicators that can help us get started:
    - Number of upvotes on an issue
    - Context from comments or customer support mentioning the number of users impacted
    - Kibana and Sentry prod logs for tracking how many users are experiencing specific errors
    - Some product groups have dashboards and other tools to track xMAU (monthly active usage) for their features. This can help us gauge how impactful a bug may be if it is affecting a feature with high usage.
- Not actively being worked on
- Not already assigned to a future milestone (unless it requires escalation)
- Not currently blocked
- Aligns with the group's current feature work and roadmap (ex: it may be more efficient to fix bugs relating to feature A together,
  while feature A is being updated in a future milestone)

We should also aim to align our decisions with the below product prioritization framework

{{% include "includes/master-prioritization-list.md" %}}

SECTION: engineering/infrastructure-platforms/developer-experience/dashboards.md

> In pursuit of a resilient, efficient, and fortified platform, our Test Platform sub-department provides instrumental
> support through the creation and maintenance of an array of dashboards. These critical tools

capture detailed test
> health across the environments, flag performance anomalies, and contribute to rigorous test infrastructure.

Dashboards

| Category | Link (Internal Only)
| Description
| DRI
|
|---|---

---|

---|

---|
| Test Health | E2E Test Status
| Comprehensive overview of test suite health and status.
|
|
| | Quarantined E2E Tests Dashboard
| Shows age and number of Quarantined tests per stage
|
|
| Test Tooling | E2E Test Execution Time
| Metrics and trends on end-to-end test execution times.
| Development Analytics
|
| | Test Flakiness (E2E Test)
| Analysis of flakiness in end-to-end tests with real-time data.
| Development Analytics
|
| | Test Flakiness (Unit, Integration & System Test)
| Statistics on flakiness in feature and unit tests.
| Engineering Productivity
|
| | Test Economy
| Insights into the cost-effectiveness of testing processes.
| Development Analytics
|
| | Test Infra Availability (E2E Test) | Dashboard indicating the current availability of E2E test infrastructure.
| Development Analytics
|
| Test Contribution | Developer Contribution To E2E Tests
| Tracking developer engagement and contributions to E2E tests.
| Test Governance
 Development Analytics |
| Performance Testing | GPT Results
| GitLab Performance Tool test results from automated reference environment pipelines

| Self-Managed Platform
|
| Self-Managed Excellence | Reference Architectures
| Reference Architecture issues created and closed per request type
| Self-Managed Platform
|
| | GitLab Environment Toolkit
| GitLab Environment Toolkit merge requests and issues per type
| Self-Managed Platform
|
| | GitLab Performance Tool
| GitLab Performance Tool merge requests and issues per type
| Self-Managed Platform
|
| Bug Tracking | Customer Bugs
| Dynamic tracking of Customer bugs
|
|
| | Bug Prioritization
| Bug prioritization based on burndown trends
|
|
| Operational Excellence | Team Operational Trends
| This dashboard is designed to give us a deeper understanding of Test Platform sub-department's operational effectiveness over time | Development Analytics
|
| Test Platform On-Call Metrics | On-call Trends
| Dashboard indicating on-call engagement
| Development Analytics
|

Contributing to the Dashboards

If you have any questions or suggestions about the dashboard, please contact the respective DRI or submit an issue in
our tracking system.

SECTION: engineering/infrastructure-platforms/developer-experience/learning-resources.md

Software Engineer in Test Focus

Books

GitLab has book clubs. Considering joining one of the upcoming book clubs or starting your own.

Mentorship Opportunities

- Ad-hoc GitLab Mentorship

Quality Engineering Manager Focus

GitLab-led Learning

- Elevate Leadership Training (open to aspiring managers starting in late FY24)

Books

- Peopleware: Productive Projects and Teams by Tom DeMarco, Tim Lister
- Drive by Daniel Pink
- Remote by Jason Fried, David Heinemeier Hansson
- It's Not About the Coffee: Lessons on Putting People First from a Life at Starbucks by Howard Behar, Janet Goldstein
- The Manager's Path: A Guide for Tech Leaders Navigating Growth and Change by Camille Fournier

Additional reading

- GitLab has book clubs. Considering joining one of the upcoming book clubs or starting your own.
- The Create Stage has recommended books for Engineering Managers.

Podcasts

- HBR Women at Work
- Coaching for Leaders

Mentorship Opportunities

- Ad-hoc GitLab Mentorship

Misc

- Software Lead Weekly (SWLW) Newsletter

SECTION: engineering/infrastructure-platforms/developer-experience/onboarding.md

The instructions here are in addition to the on-boarding issue that People Ops will assign to the team member on their first day at GitLab.

Copy the Test Platform team onboarding issue template
into a new issue in TP Team Tasks
and complete the issue.

General team resources

GitLab QA
  Testing Guide / E2E Tests
  GitLab QA Orchestrator Documentation

- GitLab QA Testing Documentation
- General testing guidelines
  - Testing standards and style guidelines
- CI infrastructure for CE and EE
  - GitLab project pipelines
  - Testing from CI
  - Review Apps
- Tests statistics
  - Redash Test Suite Statistics
- Insights dashboard
  - Quality Dashboard
  - Quality Dashboard Documentation
- Triage
  - Triage Onboarding
- QA Runners
  - QA Runner Ownership Issue
- Projects
  - gitlab-org
  - gitlab-com
  - triage-ops
- Environments
  - dev.gitlab.org
  - Production
  - Canary - gitlabcanary=true
  - Staging
  - Staging-canary - gitlabcanary=true
  - Staging-ref
  - Preprod
  - Release
  - Customer-dot
- CI environments
  - Main - Slack channel e2e-run-master
  - Performance - Slack channel gpt-performance-run
- Code Review
  - Test Platform team's code review checklists

- Slack channels

These internal Slack channels may be helpful to join.

- Department
  - infrastructure-department - general department channel
  - test-platform - general sub-department channel
  - test-platform-lounge - channel where the Test Platform sub-department hangs out and posts their weekly updates
  - test-platform-maintainers - channel for test platorm maintainers to request. Can be used to request expedited maintainer reviews when required
  - infrastructure-managers - channel to communicate and collaborate with all engineering managers in Infrastructure
  - e2e-run-master - channel with end-to-end test results on master pipeline

e2e-run-preprod - channel with end-to-end test results for run against pre.gitlab.com
e2e-run-release - channel with end-to-end test results for run against release.gitlab.net
e2e-run-staging - channel with end-to-end test results for run against staging.gitlab.com
e2e-run-production - channel with end-to-end test results for run against gitlab.com
gpt-performance-run - channel with performance testing results
quality-reports - channel with various end-to-end test metrics reports

Company

product - observe and interact with members of the product team
development - provides awareness of broken master, environment issues and other development related status items
is-this-known - find information about canary failures or bugs
questions - ask questions and see other questions (and the answers) from other GitLab Team Members
thanks - give and see thanks for the awesomeness that GitLab Team Members do
people-connect - interact with people ops

Manager

Engagement Quadrant

The engagement Quadrant is designed to help you and your direct report evaluate how they currently feel about their work.
This is not intended to be a performance evaluation tool, but rather a self-introspective mechanism to help frame the conversation.

Low knowledge & High excitement: When we are excited on starting something new but unaware of all the things needed to succeed (unknown unknowns).
Low knowledge & Low excitement: As time progresses if we haven't made progress on acquiring knowledge (sustained unknown unknowns), the excitement is also lowered. We need to expedite on attaining additional help to unblock.
High knowledge & High excitement: We have made progress on learning what is needed to be productive/proficient while feeling engaged and excited. This is the optimal state.
High knowledge & Low excitement: When we are already proficient at the current task but it may not be as challenging. We should have a discussion to identify the next area of interest.

When starting something new, the goal is to discover unknowns and learn them quickly so you can move into the High knowledge & High excitement state as fast as possible. Then keep iterating/improving so that you are still challenged and stay there.

Organizational psychology resources

Organizational psychology is the study of human behavior and motivations as it relates to work.

WorkLife podcast by Adam Grant
Heartbeat podcast by Claire Lew
HBR IdeaCast podcast
Dear HBR podcast
Know Your Team blog - most popular articles

People to follow on social media

Adam Grant - Organizational psychologist professor from Wharton
Dan Pink - author of Drive: The surprising truth about what motivates us
Camille Fournier
Claire Lew - CEO of Know Your Team.
Dan Ariely - more behavioral economics than organizational psychology

---
SECTION: engineering/infrastructure-platforms/developer-experience/project-management.md

Established Projects

Established Projects are those that have proven their value, are mature, and are fully integrated into regular department workflows. These projects are critical to the department's operations and require ongoing management and development.

- Ensure that the project is created under the respective Group within the Developer Experience Group for organizational alignment.
- Make projects Public or Internal based on the required usage needs.
- Add Ownership details for the established project by following below steps:
  - Navigate to your project and select "Settings" > "General".
  - In the General settings, find and expand the "Badges" section.
  - Enter "Maintainers" as the badge name.
  - Provide the link to the team page in the handbook. This should be the team-specific page that lists the team Slack handle, team members, and board details.
  - Use a custom badge image URL structured as suggested: https://img.shields.io/badge/maintainedby-{{teamname}}-blue. Replace {{teamname}} with the appropriate Group under Developer Experience.
  - After filling in the details, select "Add badge" to create the maintainer badge.
  - If there are multiple maintainers, create separate maintainer badges
- Adhere to GitLab's code of conduct and privacy policies.

Project Deprecation

Deprecation of a project is a significant decision and should be based on clear, objective criteria. Here are the key factors to consider:

- Decline in usage metrics over a sustained period.
- Lack of relevance to current organizational goals or technology trends.
- Existence of newer tools or platforms that effectively replace the project's functionality.

The process of deprecating a project should be methodical and transparent to all stakeholders:

- Conduct a thorough review of the project against the deprecation criteria.
- Inform all stakeholders, including project maintainers, users, and dependent teams, about the decision to deprecate.
- For an established project, ensure all valuable data and documentation are securely archived.
- Archive the project. This action will make the project read-only and prevent further changes.

If needed, it can be unarchived in the future.
- Consider deleting the project after a three-month window from the archival for established projects. Deletion can be performed immediately for personal and POC projects once the purpose is served.

Project Management

Our sub-department's work can span over 10k+ issues hence as a result, we have specific team boards for the 3 teams in Test Platform Sub-department.

Test Governance

This board shows the current ownership of workkload / issues related to Test Governance.

Development Analytics

This board shows the current ownership of workload / issues related to Development Analytics.

Developer Tooling

This board shows the current ownership of workload / issues related to Developer Tooling.

SECTION: engineering/infrastructure-platforms/developer-experience/roadmap.md

The Test Platform sub-department roadmap is divided into multiple Tracks.

Each track consists of smaller epics or issues.
Each track captures the type of work grouped by a theme. Each theme is broken down into categories and phases so they can be worked on in iterations and put on a delivery timeline.
Each Track maps directly to a track level epic. They are labeled with "~Quality" and ending with "track"

Whole department roadmap view of all tracks.

An umbrella epic Test Platform sub-department Roadmap shows the timeline of everything.
This epic is static and should not be edited. It is designed to only contain track level epics.

Roadmap Management

Linking epics

The track level epics and the Test Platform sub-department Roadmap epic are the only epics allowed to have child epics.
Each numbered list item in the roadmap should be a link either to an epic or an issue. They should be added to one of the track level epics.
Child epics that form a track should only have issues.
Epic structure:

1. Department roadmap.
1. Tracks.
1. Everything else.

Creating new epics

In the spirit of keeping things easily discoverable and reducing unnecessary epics, please refrain from creating new epics unless there are 5 or more issues created/scoped for that new epic.
Please refrain from creating new tracks level epics unless it is really nessessary.

Tracks

Coverage track (roadmap view)

Coverage track (roadmap view)

Work to improve the overall test coverage.

Testcase Management

A testcase management system to document all E2E tests and their types, and link to test reports.

1. Basic Google spreadsheet to plan type of meta data. => Done
1. Use GitLab issues as testcase management using gitlab-org/quality/testcases project.
1. Integration testcases in gitlab-org/quality/testcases with automated tests. Rspec reporter that references testcase from code so they can be updated/synced dynamically.
1. Testcase Management as GitLab Native feature. Epic

Cross-browser tests

Issues: Crossbrowser & Mobile Browser testing coverage and infrastructure

1. Basic capability to run on other browsers besides chrome (IE11, Firefox).
1. Internet Explorer, and Edge coverage.
1. Other desktop browsers.
1. Mobile browser coverage.
1. Run smoke test on soon to be new stable versions of important browsers to detect issues early.

Ecosystem tests

Epic: QA: GitLab 3rd party ecosystem testing

1. Basic tests for LDAP and SAML.
1. Github and Google OAuth tests.
1. Jenkins and Jira integration tests.

Visual-diff tests

Browser screenshot visual testing to catch visual bugs. Helps with validating layout, UX, and accessibility.

Issue: End-to-end visual regression validation.

1. Lean on 3rd party tools to get something running fast and learn from the product.
1. Basic pixel to pixel comparison implemented as part of GitLab.
1. Visual coverage smoke test for all stage groups.
1. Use visual diff as a GitLab Native feature.

Performance tests

1. Basic functional tests that creates a big issue and merge request. => Done
1. First stress test environment for on-prem customers with test runs and monitoring. => Done
1. Real 10,000 user reference architecture with customer reference traffic load testing.

Security tests

1. XSS functional tests.

Mutation tests

Issue: Mutation tests

1. Implement a POC for the ruby and JavaScript code to gather data
1. Data analysis and action items
1. Depends on the above

Test planning

Test planning process.

1. Roll out testplan format and planning process.
1. Bake in test planning as part of feature planning process.
1. Iterate and come up with a shorter version of the 10 min ACC framework.

Efficiency track (roadmap view)

Efficiency track (roadmap view)

Work that increases our efficiency and productivity.

Fault Tolerance

1. Test retry.
1. Dynamic Page Object locators validation.
1. API client automatically retries on 5xx errors.

Faster Execution

Running test faster.

1. Basic parallelization via runners.
1. Parallelization at the process level for all E2E tests, exponential cost saving of CI runners.
1. Run all tests at the same time, the whole suite takes only as long as the longest test.
1. Evaluation of a subset of tests instead of running all the E2E tests depending on what changed.

Investigate the use of running jobs when there are changes for a given path for GitLab QA jobs.

API Usage

Use API in all E2E tests. Achieve optimal test layering with API suite more than %60 of total tests.

1. Use API calls to build the most used resources (Group, Project, User) in tests that are not focused on testing these resources behaviors. => Done
1. Use API calls in login and logout for smoke tests.
1. Implement API fabrication for all the resources that support it.
1. Use API calls for setting up all resources in every E2E tests. Roll out as a standard for every new test.

Lean pyramid

Optimize test coverage across the layers of the test pyramid, to remove redundant tests and achieve higher coverage with greater efficiency.

Phase 1: TBD

Test data

Come up with standardized test data that can be seeded in all environments for productivity.

1. Test data curation, define a test datamodel which is static.
Define better project structure for ease of debugging, more readability in automated test data output, better group, project and issue naming (not just using timestamps).
1. Script to setup testdata and clean them up.

Idempotent script based on API calls (E.g. adds project if missing, uses existing if exists).

1. Setup 50% of planned test data from Phase 2 in GDK, Review Apps, Staging and Canary/Production.
1. Setup 100% of planned test data from Phase 2 in GDK, Review Apps, Staging and Canary/Production.

Test results

Work that will allow us to debug tests more easily. Includes better reporting and more informative artifacts.

1. Better readability in test output.
1. Basic HTML reporting.
1. Automated reporting into the Testcase management system.
1. Automated test name update via linking of issue ID in test code.

Test readability

1. Standard Page Object method names for click navigation.
1. TBD

Triage track (roadmap view)

Triage track (roadmap view)

Work that will help us triage issues and merge requests more efficiently.

Triage reports

1. Team level triage reports. => Done
1. Group level triage reports. => Done
1. Summary report in group reports showing the amount of bugs for that group. Divide up into quadrants of severity and priority (S/P) labels Generate 5x5 grid heat map report on existing open bugs for all group triage reports. => Done
1. Bug SLA summary report in group reports. New issue first triage SLA

Refinement

1. Basic reminder for issues and merge requests. => Done

    Merge requests that are open for a long time
    Merge requests that do not have appropriate stage, group, and type labels.
    Issues that are open for a long time (3 months / 6 months).
    Merge requests that do not have any labels or milestones.

1. Enforce one team label per merge request.
1. Automatically infer stage and group label from category labels
1. Automatically infer team label from author.
1. Automatic labelling via natural language processing of issue description.

    Investigate integrating https://gitlab.com/tromika/gitlab-issues-label-classification with GitLab Insights.

Measure track (roadmap view)

Measure track (roadmap view)

Work involving metrics that will allow us to make good data-driven decisions and report them to

stakeholders early.

Insights

1. GitLab Insights prototype with initial metrics. => Done.
1. Migrate GitLab Insights into GitLab. => Done.

Bug metrics

1. Overall creation rate of bugs. => Done.
1. Creation rate of bugs displayed in each group stage's dashboard.
1. On-prem customer incidents per month

SLA metrics

1. Mean time to resolve priority::1 and priority::2 defects.
1. New issue first triage

Releases track (roadmap view)

Releases track (roadmap view)

Work that helps in validating the release process.

Scheduling

1. Milestone refinement introduction When a milestone ends, close expired milestone and bulk reschedule unfinished work (Issues& MRs) to the next milestone => Done
1. Next iteration of closing milestones and moving issues and MRs to the next milestone

Review apps

1. Improve review apps reliability
1. Make review app a mandatory testing gate with smoke tests.
1. Shift QA tests to completely run against review apps, only orchestrated test run in the test-on-omnibus job.
1. Improve review apps usefulness, add testdata into review apps to ease testability.

SECTION: engineering/infrastructure-platforms/developer-experience/design-documents/_index.md

A design document describes a technical vision and a set of principles
that will guide various Developer Experience tool implementation, as we move forward. It acts as guardrails
to keep team aligned.

They are version controlled documents that are constantly updated with new insights and knowledge,

after every iteration, to become even more useful with time.

Contributing

At GitLab, everyone can contribute, including to our design documents. If you would like to contribute to any of these documents, feel free to:

1. Go to the source files in the repository and select the design document you wish to contribute to.
1. Create a merge request.
1. @ message both an author assigned to the design document, as listed below.

{{< engineering/design-documents-list folder="handbook/engineering/infrastructure-platforms/developer-experience/design-documents" >}}

SECTION: engineering/infrastructure-platforms/developer-experience/design-documents/_template.md

<!--
Before you start:

- Copy this file to a sub-directory and call it index.md for it to appear in
  the design documents list.
- Remove comment blocks for sections you've filled in.
  When your document ready for review, all of these comment blocks should be
  removed.

To get started with a document you can use this template to inform you about
what you may want to document in it at the beginning. This content will change
/ evolve as you move forward with the proposal. You are not constrained by the
content in this template. If you have a good idea about what should be in your
document, you can ignore the template, but if you don't know yet what should
be in it, this template might be handy.

- Fill out this file as best you can. At minimum, you should fill in the
  "Summary", and "Motivation" sections. These can be brief and may be a copy
  of issue or epic descriptions if the initiative is already on Product's
  roadmap.
- Create a MR for this document. Assign it to an Architecture Evolution
  Coach (i.e. a Principal+ engineer).
- Merge early and iterate. Avoid getting hung up on specific details and
  instead aim to get the goals of the document clarified and merged quickly.
  The best way to do this is to just start with the high-level sections and fill
  out details incrementally in subsequent MRs.

Just because a document is merged does not mean it is complete or approved.
Any document is a working document and subject to change at any time.

When editing documents, aim for tightly-scoped, single-topic MRs to keep discussions focused. If you disagree with what is already in a document, open a new MR with suggested changes.

If there are new details that belong in the document, edit the document. Once a feature has become "implemented", major changes should get new blueprints.

The canonical place for the latest set of instructions (and the likely source of this file) is content/handbook/engineering/architecture/design-documents/template.md.

Document statuses you can use:

- "proposed"
- "accepted"
- "ongoing"
- "implemented"
- "postponed"
- "rejected"

-->

<!-- Design Documents often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

<!--
Don't add a h1 headline. It'll be added automatically from the title front matter attribute.

For long pages, consider creating a table of contents.
-->

Summary

<!--
This section is very important, because very often it is the only section that will be read by team members. We sometimes call it an "Executive summary", because executives usually don't have time to read entire documents like this. Focus on writing this section in a way that anyone can understand what it says, the audience here is everyone: executives, product managers, engineers, wider community members.

A good summary is probably at least a paragraph in length.
-->

Motivation

<!--

This section is for explicitly listing the motivation, goals and non-goals of
this document. Describe why the change is important, all the opportunities,
and the benefits to users.

The motivation section can optionally provide links to issues that demonstrate
interest in a document within the wider GitLab community. Links to
documentation for competing products and services is also encouraged in cases
where they demonstrate clear gaps in the functionality GitLab provides.

For concrete proposals we recommend laying out goals and non-goals explicitly,
but this section may be framed in terms of problem statements, challenges, or
opportunities. The latter may be a more suitable framework in cases where the
problem is not well-defined or design details not yet established.
-->

Goals

<!--
List the specific goals / opportunities of the document.

- What is it trying to achieve?
- How will we know that this has succeeded?
- What are other less tangible opportunities here?
-->

Non-Goals

<!--
Listing non-goals helps to focus discussion and make progress. This section is
optional.

- What is out of scope for this document?
-->

Proposal

<!--
This is where we get down to the specifics of what the proposal actually is,
but keep it simple! This should have enough detail that reviewers can
understand exactly what you're proposing, but should not include things like
API designs or implementation. The "Design Details" section below is for the
real nitty-gritty.

You might want to consider including the pros and cons of the proposed solution so that they can be
compared with the pros and cons of alternatives.
-->

Design and implementation details

<!--
This section should contain enough information that the specifics of your
change are understandable. This may include API specs (though not always
required) or even code snippets. If there's any ambiguity about HOW your
proposal will be implemented, this is the place to discuss them.

If you are not sure how many implementation details you should include in the
document, the rule of thumb here is to provide enough context for people to
understand the proposal. As you move forward with the implementation, you may
need to add more implementation details to the document, as those may become
valuable context for important technical decisions made along the way. A
document is also a register of such technical decisions. If a technical
decision requires additional context before it can be made, you probably should
document this context in a document. If it is a small technical decision that
can be made in a merge request by an author and a maintainer, you probably do
not need to document it here. The impact a technical decision will have is
another helpful information - if a technical decision is very impactful,
documenting it, along with associated implementation details, is advisable.

If it's helpful to include workflow diagrams or any other related images.
Diagrams authored in GitLab flavored markdown are preferred. In cases where
that is not feasible, images should be placed under images/ in the same
directory as the index.md for the proposal.
-->

Alternative Solutions

<!--
It might be a good idea to include a list of alternative solutions or paths considered,
although it is not required. Include pros and cons for
each alternative solution/path.

"Do nothing" and its pros and cons could be included in the list too.
-->

SECTION: engineering/infrastructure-platforms/developer-experience/design-documents/browser_performance_testing/_index.md

{{< engineering/design-document-header >}}

[[TOC]]

Executive Summary

This blueprint outlines a comprehensive approach to implementing browser performance testing at GitLab with a dual focus: "shift-left" testing at the Merge Request (MR) level to detect performance issues early and "shift-right" testing against production environments to validate performance in real-world conditions. The approach leverages Sitespeed.io as the core

technology while addressing challenges around test environment consistency, data seeding, and meaningful metrics.

```
mermaid
flowchart LR
   subgraph "Shift-Right"
      SR1[Production Testing on gitlab.com]
      SR2[Grafana Dashboards]
      SR3[Long-term Trend Analysis]
      SR4[Issue Notifications]
   end

   subgraph "Shift-Left"
      SL1[MR Pipeline Integration]
      SL2[Baseline Comparison]
      SL3[Performance Widget]
      SL4[Regression Detection]
   end

   SIT[Browser Performance Testing]

   SIT --> SR1
   SIT --> SL1

   SR1 --> SR2
   SR2 --> SR3
   SR3 --> SR4

   SL1 --> SL2
   SL2 --> SL3
   SL3 --> SL4

   classDef shift-right fill:6BB3F2,stroke:333,stroke-width:1px,color:white,rx:5px;
   classDef shift-left fill:F28C6B,stroke:333,stroke-width:1px,color:white,rx:5px;
   classDef core fill:8566CC,stroke:333,stroke-width:1px,color:white,rx:10px;

   class SR1,SR2,SR3,SR4 shift-right;
   class SL1,SL2,SL3,SL4 shift-left;
   class SIT core;
```

Background

The Frontend team previously utilized a Sitespeed setup (GitLab.org/frontend/sitespeed-measurement-setup) for performance testing, but this system became obsolete. The Performance Enablement team has been tasked with reviving and improving this capability as part of ongoing efforts to enhance application performance across GitLab.

In parallel, the DevEx team has been running the GitLab Browser Performance Testing (GBPT) solution (gitlab-org/quality/performance-sitespeed) against 1k self-managed instances and

staging environments. This existing solution provides valuable performance data but is primarily focused on controlled self-managed environments rather than testing against production or at the MR level.

Both server-side performance issues (detected by GitLab Performance Tool - GPT) and client-side performance issues (detected by GBPT) are currently identified primarily through nightly runs. This means that issues are often caught after they've been merged but before they're released to self-managed instances. However, for gitlab.com, these issues may persist for some time until fixed. A more proactive approach is needed to catch these issues earlier in the development lifecycle.

This blueprint aligns with broader performance engineering initiatives at GitLab, including the proposed Performance Results Datastore that will centralize all performance test results for advanced analytics and trend analysis. The browser performance testing capabilities outlined here will feed data into this central repository, enabling more sophisticated analysis and visualization of client-side performance metrics across environments and GitLab versions.

Current Challenges

1. Late Detection: Performance issues are often discovered only after reaching production
2. Environment Inconsistency: Results vary across different environments, making baseline comparisons difficult
3. Test Data Variability: Performance metrics are sensitive to test data volume and complexity
4. Delayed Feedback: The team often has to review hundreds of MRs to identify performance regressions
5. Resource Constraints: Spinning up dedicated test environments for each MR is resource-intensive
6. Cells Architecture: Future transitions to a Cells architecture will complicate performance testing approaches
7. Fragmented Solutions: Current solutions (obsolete Frontend Sitespeed and GBPT for 1k instances) operate independently, limiting cross-environment comparison and insight sharing
8. Limited MR-Level Testing: Existing solutions focus on environment-level testing rather than providing immediate feedback at the code change level
9. Authentication Requirements: Some endpoints require user authentication, necessitating framework capabilities to handle login flows and maintain selectors
10. Page Interaction Complexity: Certain tests require complex interactions with the page, requiring teams to build and maintain custom test scripts

Strategic Approach

We propose a dual-track implementation strategy leveraging GitLab's Runway platform for infrastructure:

Track 1: Shift-Left (MR-Level Testing)

Implement browser performance testing at the MR level to catch performance regressions before they reach production. This approach provides developers with immediate feedback on how their changes impact client-side performance.

Data Flow Architecture for MR Integration

```
mermaid
graph TD
    A[MR Pipeline] -->|Run tests| B[Sitespeed.io]
    B -->|Generate metrics| C[Result JSON]
    C -->|Query baseline| D[Performance Results Datastore]
    D -->|Return baseline| E[Comparison Logic]
    C -->|Input current results| E
    E -->|Generate comparison| F[MR Widget]
    C -->|Store results| D
    G[Master Tests] -->|Run tests| B
    H[Grafana] -->|Visualize trends| D
```

Track 2: Shift-Right (Production Monitoring)

Deploy the Sitespeed.io Online setup via Runway on GCP VMs to enable regular performance testing against production environments (gitlab.com and staging.gitlab.com). This establishes performance baselines and monitors long-term trends, satisfying the Frontend team's requirements while providing valuable comparative data.

Data Flow Architecture for Production Monitoring

```
mermaid
flowchart LR
    PROD[gitlab.com] -->|Tests| SS[Sitespeed.io]

    SS -->|Store| DB[(Databases)]
    DB -->|Visualize| GRAF[Grafana]
    DB -->|Threshold Check| ALERT[Alerts]

    GRAF -->|View| TEAMS[Slack]
    ALERT -->|Notify| TEAMS

    classDef prod fill:6BB3F2,stroke:333,stroke-width:1px,color:white;
    classDef tool fill:F28C6B,stroke:333,stroke-width:1px,color:white;
    classDef storage fill:67B96A,stroke:333,stroke-width:1px,color:white;
    classDef viz fill:8566CC,stroke:333,stroke-width:1px,color:white;
    classDef users fill:A9A9A9,stroke:333,stroke-width:1px,color:white;

    class PROD prod;
    class SS tool;
    class DB storage;
    class GRAF,ALERT viz;
    class TEAMS users;
```

Integration with Existing Solutions

Leverage and integrate with the existing GitLab Browser Performance Testing (GBPT) solution (gitlab-org/quality/performance-sitespeed) that currently runs against 1k self-managed

instances and staging environments. This integration will:

1. Provide comparative baselines between controlled self-managed environments and production
2. Allow correlation of performance trends across different environment types
3. Create a more comprehensive picture of browser performance across GitLab's deployment models
4. Reduce duplication of effort while maximizing the value of existing investments

Note: After Phase 1 implementation, we will reassess whether maintaining separate GBPT testing alongside the new implementation is necessary or if consolidation would be more efficient.

Implementation Plan

Phase 1: Core Infrastructure & Production Monitoring Setup

1. Deploy Sitespeed.io using Runway Architecture
   Leverage Runway (GitLab's internal platform) for deploying the Sitespeed.io infrastructure
   Configure the Sitespeed Online components on GCP VMs (as mentioned in the original issue)
   Set up the following components as defined in the Runway architecture:
     KeyDB for caching and job queuing
     PostgreSQL for test metadata and configuration storage
     Minio (or equivalent S3 storage) for result storage
     Sitespeed.io test runners (version 36.4.1)
     Online GUI and API components
   Implement security best practices:
     Change all default passwords
     Configure Basic Auth for administration
     Set secret keys for API access
     Implement domain regex restrictions
2. Define Key Performance Metrics
   First Contentful Paint (FCP)
   Largest Contentful Paint (LCP)
   Total Blocking Time (TBT)
   Speed Index (SI)
   Layout Volatility (LVC)
   Transfer Size (TFR SIZE)
   Performance Score
   Accessibility (AXE integration)
3. Set Up Monitoring and Visualization
   Configure Grafana dashboards for viewing performance metrics
   Integrate with the Performance Results Datastore for centralized analytics
   Configure InfluxDB storage bucket for time-series performance data
   Set up alert thresholds for critical performance degradations
4. Production Test Configuration
   Configure tests for gitlab.com environments
   Set up competitor comparison tests
   Implement login capabilities for authenticated page testing
   Configure data retention policies for test results
   Integrate with existing GBPT (gitlab-org/quality/performance-sitespeed) data pipelines
   Establish correlation between 1k self-managed instance results and production metrics
5. Results Visualization & Reporting

- Set up Grafana dashboards for production metrics
- Create comparison views between environments
- Implement trend analysis for long-term monitoring
- Configure webhooks for notifications to issues/slack channels
- Create regular performance reports
- Enable cross-reference visualization between GBPT results and the new Sitespeed setup

Phase 2: MR-Level Integration

1. Leverage Existing CI/CD Resources
   - Utilize existing CI/CD review apps or CNG instances rather than spinning up new environments
   - Execute Sitespeed tests after E2E tests complete on the same environment
   - Consider building dedicated environments with CNG orchestrator if sharing with E2E tests introduces result variability
2. Baseline Establishment
   - Create master branch baselines for each key page/workflow
   - Document baseline performance in a central location for reference
   - Implement automatic baseline updates on a regular cadence
3. MR Integration
   - Create a CI job that executes browser performance tests
   - Implement comparison logic between MR results and baseline
   - Generate browser-performance.json for MR widget display
   - Add support for selective test execution to run only relevant tests based on changed files
4. Alerting System
   - Define threshold values for performance degradation
   - Create MR comments for performance issues
   - Implement Slack notifications for significant regressions

Phase 3: Refinement & Expansion

1. User Journey Support
   - Implement framework for defining custom user journeys
   - Support feature flag testing via URL parameters
   - Allow teams to create specialized workflows
2. Documentation & Self-Service
   - Create operational guides for teams
   - Provide documentation on adding custom tests
   - Establish support channels for questions
3. Future Architecture Support
   - Plan for Cells architecture compatibility
   - Develop Cell-specific testing strategies
   - Create a strategy for Cells-aware production monitoring including:
     - Cell-specific test configurations and baselines
     - Inter-Cell interaction testing
     - Cell-attribution for performance metrics

Technical Implementation Details

Environment Strategy

1. MR Testing:
   Use existing review apps or CNG instances after E2E tests complete
   Test against a controlled, consistently seeded environment
   Compare directly against master branch baseline on the same environment type
   Leverage dynamic baselines from the Performance Results Datastore
2. Production Testing:
   Run against specific endpoints on gitlab.com and staging.gitlab.com
   Use a 1K reference architecture for comparative baseline (aligning with existing GBPT approach)
   Schedule at regular intervals to identify long-term trends
   Be mindful of future Cells architecture considerations when testing against production
3. Existing GBPT Integration:
   Correlate results with existing GBPT 1k self-managed instance tests
   Use GBPT results as reference points for controlled environment performance
   Share test definitions and page scenarios where applicable
   Allow cross-comparison between controlled environments and production/MR results

Performance Data Storage

1. Time-Series Data:
   Store browser performance metrics in InfluxDB as part of the Performance Results Datastore
   Tag data with relevant metadata (GitLab version, environment, test type, etc.)
   Set appropriate retention policies for historical analysis
2. Baseline Management:
   Store baselines in both InfluxDB (for trend analysis) and JSON files (for CI access)
   Update baselines automatically based on statistical analysis of recent results
   Support version-specific baselines for different GitLab releases

Data Seeding Approach

1. Consistent Test Data:
   Implement a data seeding strategy based on identified requirements:
     Quick seeding process to minimize test setup time
     Stable and consistent data across test runs
     Developer-extensible for adding specialized test data
   Evaluate options referenced in the GitLab Seeding Options Runbook
   Pre-load test data into the environment
   Consider pre-built Docker images with consistent data
2. Test Pages Selection:
   Identify high-impact pages for testing (MR with loaded content, issue pages, etc.)
   Include pages with complex UI components
   Test both logged-out and logged-in views

Threshold Management

Implement dynamic thresholds based on baseline + percentage variance
Start with industry standards:
  LCP: \< 2.5s (good), \< 4s (needs improvement), \> 4s (poor)
  TBT: \< 200ms (good), \< 600ms (needs improvement), \> 600ms (poor)
Allow team-specific threshold customization

Rollout Strategy

1. Pilot with Frontend Team:
   Start with the Source Code Management team who requested the revival
   Focus on a small set of critical pages
   Gather feedback and refine approach
2. Expand to Other Teams:
   Onboard additional teams with similar needs
   Provide documentation and support
   Collect use cases and implementation feedback
3. Organization-wide Deployment:
   Make available to all development teams
   Incorporate into developer workflow documentation
   Track adoption and effectiveness

Success Metrics

The success of this implementation will be measured by:

1. Early Detection: Number of performance issues caught at MR level vs. production
2. Response Time: Reduction in time to identify and fix performance regressions
3. Developer Adoption: Number of teams actively using browser performance testing
4. Performance Trends: Improvement in key performance metrics over time
5. Reliability: Consistency of test results across environments
6. Data Accessibility: Adoption of performance data visualizations by engineering teams
7. Integration Effectiveness: Successful integration with the Performance Results Datastore
8. Decision Impact: Number of data-driven decisions made using performance insights

Maintenance Model

To ensure long-term success of this initiative, the following maintenance model will be implemented:

1. Performance Enablement Team Responsibilities:
   Maintain the core framework and infrastructure
   Ensure integration with CI/CD pipeline
   Provide support for the baseline comparison logic
   Maintain integration with the Performance Results Datastore
   Update core components as needed
2. Frontend Teams Responsibilities:
   Create and maintain specific tests for their areas
   Monitor alerts for their components
   Address performance regressions in their code
   Adjust thresholds as needed for their components
   Contribute to test coverage expansion
3. Shared Responsibilities:
   Documentation updates
   Onboarding new teams
   Performance metrics definition
   Test data management

This clear division of responsibilities ensures the sustainability of the solution while maximizing team ownership of performance outcomes.

Conclusion

This browser performance testing blueprint outlines a comprehensive approach to addressing both immediate needs for client-side performance testing and long-term strategic goals for early detection of issues. By implementing both shift-left and shift-right testing, we can provide developers with immediate feedback while maintaining visibility into production performance.

The phased implementation approach allows for incremental delivery of value, starting with the most critical components and gradually expanding capabilities. This blueprint addresses the key challenges identified by stakeholders while providing a scalable foundation for future performance testing needs.

References

Sitespeed.io Documentation
GitLab Browser Performance Testing Documentation
Web Vitals Initiative
GitLab Issue 17: Revive the Sitespeed setup
GitLab Epic 146: Enable GitLab Browser Performance Tool to be used in MRs
GitLab Epic 134: Production Monitoring for Browser Performance using Sitespeed
Performance Results Datastore Blueprint
Shift Left and Right Performance Testing
GitLab Performance Tool (GPT)
GitLab Browser Performance Testing (GBPT)
Reference Architecture Test Environment Details
Runway Documentation
GitLab Seeding Options Runbook

SECTION: engineering/infrastructure-platforms/developer-experience/design-documents/component_performance_testing/_index.md

{{< engineering/design-document-header >}}

[[TOC]]

Glossary

Term	Definition
GPT	GitLab Performance Toolkit
Real world scenario	Environment replicating production with representative data
Component	Standalone element of GitLab architecture (e.g. Gitaly, AI Gateway etc.)

Executive Summary

This blueprint outlines a comprehensive approach to implementing component-level performance testing at GitLab, enabling teams to detect performance issues earlier in the development lifecycle ("shift-left"). The approach leverages containerization and automated testing to provide insights on individual component performance metrics as well as providing immediate feedback on performance impacts of code changes at the Merge Request level.

Component Testing Tool Architecture

```
mermaid
flowchart TD
    %% Force vertical arrangement with a clearer structure

    %% Component Repository
    subgraph CR["Component Repository"]
        repoStructure["Required Directory Structure:<br/>performance-test/<br/>⋯ k6-test/
(test files)<br/>⋯ setup/ (docker-compose.yml)"]:::highlight

        A1[Developer MR]
        A2[MR Pipeline Triggered on Commit]
        A3[dotenv-vars Job]
        A4[tests:performance Job]

        A1 --> A2 --> A3 --> A4

        note1[Saves test file and setup<br/>file as artifacts]:::note
        A3 --- note1
    end

    %% Connection point - this must be outside both subgraphs
    A4 --->|"Trigger Multi-Project Pipeline"| B

    %% Component Performance Test Tool Pipeline
    subgraph CPTT["Component Performance Test Tool Pipeline"]
        B[CPT Pipeline Job]
        C["·· GCP instance:<br/>Component Container"]
        D["· Test Runner (K6)"]
        E["·· InfluxDB"]
        F["· Performance Report"]
        G["· Baseline"]
        I["· Grafana Dashboard"]

        B -->|"Spins up"| C
        B -->|"Spins up"| D
        D -->|"Execute Tests"| C
        D -->|"Collect Metrics"| E
        B -->|"Generate"| F
        F -->|"Compare with"| G
        E -->|"Visualize"| I
    end
```

```
%% Results feedback loop - must be drawn last
F -->|"Post/Update result as comment to MR"| A1

%% Style definitions
classDef componentRepo fill:f9f9f9,stroke:fc6d26,stroke-width:2px;
classDef cptTool fill:f0f0ff,stroke:4285f4,stroke-width:2px;
classDef note fill:ffffcc,stroke:999,stroke-width:1px,stroke-dasharray: 5 5;
classDef highlight fill:e6f7ff,stroke:1890ff,stroke-width:2px;
%% Dark theme compatible subgraph titles
classDef subgraphTitle fill:f9f9f9,stroke:fc6d26,stroke-width:2px,color:333333;
classDef cptTitle fill:f0f0ff,stroke:4285f4,stroke-width:2
```